



BSc (Hons) in **Information Technology**
Specialized in AI
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{ 'wall_ahead': True/False, 'food_here': True/False }` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent must store an action for every possible sequence of percepts. As the agent's lifetime increases, the number of percept sequences grows exponentially, making the lookup table unimaginably large. This would require an impractical amount of memory and computation. So, a table-driven approach cannot scale to real-world or complex environments like chess.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Condition: if percept["food_here"] --> Agent senses food in its current cell.

Action: return "Suck" --> Agent decides to consume the food.

Condition: elif percept["wall_ahead"] --> Agent senses a wall or the boundary of the grid directly in front of it.

Action: return "TurnLeft" --> Agent decides to rotate to avoid hitting the wall

Otherwise (there is no food here, and the path ahead is clear)

Action: return "MoveForward" --> Agent decides to step forward into the empty space

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The `SimpleReflexAgent` became stuck in an infinite loop because it operates in a partially observable environment and has no percept history.

The agent can only sense whether there is a wall ahead or food at its current position. It cannot determine where it has previously been or recognize that it is repeating the same sequence of actions.

Since the agent has no memory of past locations or actions, every time it returns to the same position it receives the same percept and chooses the same action again. This causes it to cycle indefinitely instead of finding an alternative path. The combination of partial observability and the absence of percept history prevents the agent from escaping repetitive behavior.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Sensor Model - The sensor model helps the agent understand how its own actions change its relationship with the world. In the code, the `get_percept()` method provides sensor information such as: `wall_ahead`, `food_here`, `collision`, `score`. The `ModelBasedAgent` uses these percepts to update its internal state. If a wall is detected ahead, it updates its model to avoid moving into that direction. If food is detected, it performs the "Suck" action and updates its internal representation after collecting the food.

Transition Model - The transition model describes how the world changes after an action. In the code, the transition model is implemented by updating the internal state after each action. For example, when the agent moves forward or turns left, it updates its stored position, facing direction, or visited cells. This enables the agent to predict the result of its actions instead of relying only on the current percept

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING